# Project Nebula: Stage One Engineering Design Manual & Architecture Guidebook

## Overview

This manual outlines the stage one engineering design and technical specification for **Project Nebula**, a hybrid desktop workstation merging a classical 64-bit/32-bit ChromeOS host environment with a localized, actively cooled **2-qubit Quantum Processing Unit (QPU)**.

---

## Section 1: System Architecture & Hardware Topology

The Nebula workstation adopts a **Host-Coprocessor Heterogeneous Architecture**, separating standard OS duties from sub-nanosecond quantum execution loops.

```
None
[Userspace Quantum Runtime / Q#]
             | (ioctl / mmap zero-copy)
             ▼
[Linux Kernel: qpu_cxl.ko Driver]
             |
             ├──> CXL.io (Control Registers & Telemetry)
             └──> CXL.mem (Quantum Execution Queue Ring Buffer)
                            |
                            ▼
          [Room-Temperature FPGA & Microwave Synthesizer]
                            |
                            ▼ (Coaxial Feedlines / RF Shielding)
            [Cryo-QPU Stage: 2-Qubit Superconducting Transmon]
```

### 1.1 The Host Controller & Fallback Layer

- **Classical Host:** A high-performance ARM64/x86-64 SoC equipped with a hardware-level 32-bit/64-bit instruction translation layer running an optimized desktop ChromeOS build.
- **Fallback State Machine:** A real-time watchdog loop monitoring the QPU status. If a decoherence event or thermal spike occurs, execution is instantly redirected to the classical CPU execution units without crashing the application thread.

### 1.2 Thermal & Cryogenic Integration Routes

Depending on the engineering trade-off between power consumption and form factor, Nebula integrates one of two primary cooling pathways:

1. **Route A: Closed-Loop Micro-Cryocooler (Selected for Desktop)**
    - **Mechanism:** Miniature pneumatic flexure-bearing Stirling-Joule-Thomson hybrid compressor.
    - **Thermal Envelope:** Maintains the primary transmon substrate at `$\le 100\,\text{mK}$` on a continuous 70W power draw.
    - **Vibration Isolation:** Mounted via elastomeric dampening dampers to eliminate micro-phonic noise and preserve qubit coherence times ($T_1/T_2$).
2. **Route B: Coaxial Feedline & RF Shielding Layout**
    - **Routing:** Superconducting coaxial niobium-titanium lines run from the room-temperature CXL controller down through multi-stage thermal radiation shields (30K stage, 4K stage, down to the 100mK mixing chamber).
    - **Attenuation & Filtering:** Microwave attenuators at each thermal stage minimize thermal noise leakage into the quantum core.

---

## Section 2: Low-Level CXL Driver Implementation

The operating system interfaces with the hardware through a modular Linux character driver (`qpu_cxl.ko`) that exposes zero-copy ring buffers and custom control commands.

```C/C++
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>
#include <linux/mm.h>
#include <linux/io.h>
#include <linux/pci.h>

#define MODULE_NAME "qpu_cxl_drv"
#define QPU_IOC_MAGIC 'q'

/* Custom ioctl commands */
#define QPU_IOCTL_START_EXEC    _IO(QPU_IOC_MAGIC, 1)
#define QPU_IOCTL_RESET_STATE   _IO(QPU_IOC_MAGIC, 2)
#define QPU_IOCTL_GET_STATUS    _IOR(QPU_IOC_MAGIC, 3, struct qpu_status_t)
```

```
struct qpu_gate_desc {
    __u32 gate_type;    /* e.g., Hadamard, CNOT, Phase */
    __u32 target_qubit; /* 0 or 1 for a 2-qubit system */
    __u64 phase_angle;  /* Fixed-point phase parameter */
    __u64 timestamp;    /* Hardware dispatch tick */
};

struct qpu_status_t {
    __u32 state_flags;     /* Bit 0: Ready, Bit 1: Decoherence Warn, Bit 2: Fault
*/
    __u32 current_temp_mk; /* Temperature in millikelvin */
    __u64 execution_count;
};

struct qpu_device_data {
    resource_size_t cxl_reg_phys;
    void __iomem *cxl_reg_base;
    struct cdev cdev;
    struct class *device_class;
    dev_t dev_num;
};

static struct qpu_device_data qpu_dev;

/* Memory mapping routine: Exposes CXL memory ring buffer to userspace (zero-copy)
*/
static int qpu_mmap(struct file *filp, struct vm_area_struct *vma)
{
    unsigned long vsize = vma->vm_end - vma->vm_start;

    if (vsize > SZ_64M)
        return -EINVAL;

    vma->vm_page_prot = pgprot_writecombine(vma->vm_page_prot);

    if (remap_pfn_range(vma, vma->vm_start,
                        qpu_dev.cxl_reg_phys >> PAGE_SHIFT,
                        vsize, vma->vm_page_prot)) {
        return -EAGAIN;
    }

    return 0;
}

/* Custom ioctl handler for hardware control */
static long qpu_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
```

```
    struct qpu_status_t status;
    int err = 0;

    if (_IOC_TYPE(cmd) != QPU_IOC_MAGIC)
        return -ENOTTY;

    switch (cmd) {
    case QPU_IOCTL_START_EXEC:
        iowrite32(0x1, qpu_dev.cxl_reg_base + 0x00);
        break;

    case QPU_IOCTL_RESET_STATE:
        iowrite32(0x3, qpu_dev.cxl_reg_base + 0x04);
        break;

    case QPU_IOCTL_GET_STATUS:
        status.state_flags = ioread32(qpu_dev.cxl_reg_base + 0x08);
        status.current_temp_mk = ioread32(qpu_dev.cxl_reg_base + 0x0C);
        status.execution_count = ioread64(qpu_dev.cxl_reg_base + 0x10);

        if (copy_to_user((void __user *)arg, &status, sizeof(status)))
            err = -EFAULT;
        break;

    default:
        err = -ENOTTY;
        break;
    }

    return err;
}

static int qpu_open(struct inode *inode, struct file *filp) { return 0; }
static int qpu_release(struct inode *inode, struct file *filp) { return 0; }

static const struct file_operations qpu_fops = {
    .owner          = THIS_MODULE,
    .open           = qpu_open,
    .release        = qpu_release,
    .unlocked_ioctl = qpu_ioctl,
    .mmap           = qpu_mmap,
};

static int __init qpu_driver_init(void)
{
    // Character device registration and CXL MMIO mapping initialization logic
    return 0;
}
```

```
static void __exit qpu_driver_exit(void)
{
    // Cleanup routines
}

module_init(qpu_driver_init);
module_exit(qpu_driver_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Quantum Systems Engineering Group");
MODULE_DESCRIPTION("Stage One CXL Driver for Hybrid Desktop QPU Workstation");
```

---

## Section 3: Stage One Verification & Next Steps

1. **Simulation Phase:** Validate the CXL memory-mapping protocol and ring buffer queue handling using QEMU-based CXL device emulation.
2. **Thermal Prototyping:** Benchmark the micro-stirling cryocooler vibration spectrum against acoustic dampening mounts.
3. **Runtime Integration:** Bind the `qpu_cxl` driver calls into the ChromeOS hybrid development runtime environment.